

对于OBS部分的分析

```
# =====  
# 1. OBSERVATION PHASE  
# =====  
# Get robot observation  
obs = robot.get_observation()  
  
# Applies a pipeline to the raw robot observation, default is IdentityProcessor  
obs_processed = robot_observation_processor(obs)  
  
if policy is not None or dataset is not None:  
    observation_frame = build_dataset_frame(dataset.features, obs_processed, prefix="ob
```

1. 获取机器人观测数据

so101_follower 的 `get_observation()` 函数获取机器人观测数据,在 `robots/so101_follower/so101_follower.py` 中实现

1.1. 获取臂观测数据

```
def get_observation(self) -> dict[str, Any]:
    """
```

获取当前机器人臂的完整观测状态 (observation)。

该方法负责同步读取机械臂的所有关节位置 (Present Position),
并从所有配置的相机异步捕获最新图像帧。

返回的字典将用于构建 LeRobot 数据集中的 observation 部分,
供 VLA (Vision-Language-Action) 模型推理使用。

返回:

dict[str, Any]: 观测字典, 包含:

- "xxx.pos": 每个电机的当前位置 (单位通常为脉冲或弧度)
- "<camera_key>": 各相机名称对应的最新图像帧 (numpy.ndarray, BGR 格式)

```
"""
```

```
# 检查机器人是否已建立连接 (串口/总线是否打开)
```

```
if not self.is_connected:
```

```
    raise DeviceNotConnectedError(f"{self} 未连接, 无法读取观测数据。")
```

```
obs_dict = {} # 初始化观测字典
```

```
# =====
```

```
# 1. 同步读取机械臂关节位置
```

```
start = time.perf_counter() # 记录读取开始时间, 用于性能监控
```

```
# 通过总线 (Feetech 协议) 同步读取所有电机的 "Present_Position"
```

```
# 返回格式通常为 {motor_name: position_value, ...}
```

```
raw_positions = self.bus.sync_read("Present_Position")
```

```
# 重命名键名, 加上 ".pos" 后缀, 符合 LeRobot 标准 observation 命名规范
```

```
# 例如: motor1 -> motor1.pos
```

```
obs_dict = {f"{motor}.pos": val for motor, val in raw_positions.items()}
```

```
# 计算读取耗时 (毫秒), 用于调试总线通信性能
```

```
dt_ms = (time.perf_counter() - start) * 1e3
```

```
logger.debug(f"{self} 读取关节状态耗时: {dt_ms:.1f}ms")
```

```
# =====
```

```
# 2. 异步读取所有相机图像
```

```
# =====
```

```
# 遍历配置的所有相机 (self.cameras 为 dict: {"cam_name": CameraInstance, ...})
```

```
for cam_key, cam in self.cameras.items():
    start = time.perf_counter() # 记录单相机读取开始时间

    # 使用异步读取方式，避免阻塞主控循环，提高实时性与帧率稳定性
    # async_read() 会从后台线程获取最新帧，最多等待一定超时时间
    obs_dict[cam_key] = cam.async_read()

    # 计算单相机读取耗时，用于监控相机性能瓶颈
    dt_ms = (time.perf_counter() - start) * 1e3
    logger.debug(f"{self} 读取相机 {cam_key} 耗时: {dt_ms:.1f}ms")

# 返回完整的观测字典，供 record_loop 或策略推理使用
return obs_dict
```

1.2. camera异步读取

cameras/opencv/camera_opencv.py 里面有 async_read 函数

```
def async_read(self, timeout_ms: float = 200) -> np.ndarray:
```

```
    """
```

异步读取最新的可用图像帧。

该方法从后台读取线程中获取最近捕获的图像帧。它不会直接阻塞等待相机硬件，但会最多等待 `timeout_ms` 毫秒，直到后台线程提供一帧新图像。

参数：

`timeout_ms (float)`：等待新帧的最大时间（毫秒）。默认为 `200ms (0.2秒)`。

返回：

`np.ndarray`：最新的图像帧，以 NumPy 数组形式返回，形状为 `(height, width, channels)`，已经根据相机配置进行处理（如颜色格式转换、裁剪等）。

异常：

`DeviceNotConnectedError`：如果相机未连接。

`TimeoutError`：如果在指定超时时间内没有收到新帧。

`RuntimeError`：如果发生内部意外错误。

```
    """
```

```
# 检查相机是否已连接
```

```
if not self.is_connected:
```

```
    raise DeviceNotConnectedError(f"{self} is not connected.")
```

```
# 如果后台读取线程尚未启动或已停止，则自动启动它
```

```
if self.thread is None or not self.thread.is_alive():
```

```
    self._start_read_thread()
```

```
# 等待新帧事件，最多等待 timeout_ms 毫秒（转换为秒）
```

```
if not self.new_frame_event.wait(timeout=timeout_ms / 1000.0):
```

```
    # 检查读取线程是否仍在运行，用于诊断问题
```

```
    thread_alive = self.thread is not None and self.thread.is_alive()
```

```
    raise TimeoutError(
```

```
        f"Timed out waiting for frame from camera {self} after {timeout_ms} ms. "
```

```
        f"Read thread alive: {thread_alive}."
```

```
    )
```

```
# 获取最新帧（线程安全，使用锁保护）
```

```
with self.frame_lock:
```

```
    frame = self.latest_frame
```

```
    # 清除新帧事件标志，避免重复触发
```

```
    self.new_frame_event.clear()
```

```
# 确保帧不为 None
```

```
if frame is None:
```

```
        raise RuntimeError(f"Internal error: Event set but no frame available for {self}")

    # 返回获取到的帧
    return frame
```

这边已经使用了异步读取，因为它不真正等待相机硬件捕获新的一帧，而是**后台线程已经提前持续从相机拉帧并缓存最新的一帧** (latest_frame)。

主线程调用 `async_read()` 时，只做两件事：

1. 最多等 200ms 看是否有新帧到来（实际几乎总是立刻就有）。
2. 用锁复制出缓存的最新帧指针。（读的主线程花的时间主要基本都在这）

2. 观测数据后处理

`obs_processed` 需要进行一个处理才能得到，调用了 `processor/pipeline.py` 里面的 `DataProcessorPipeline` 的 `__call__` 函数

```
def __call__(self, data: TInput) -> TOutput:
    transition = self.to_transition(data) # 将 obs 转为内部 EnvTransition 格式 “外部输入 →
    transformed_transition = self._forward(transition) # 通过所有 steps 处理 处理核心逻辑
    return self.to_output(transformed_transition) # 处理后中间格式 → 转为最终输出
```

LeRobot 使用管道架构（多个步骤串联）来处理数据（如观测 `obs`）

管道支持钩子：可在每步前后插自定义逻辑（如调试、日志），不改核心代码。

支持切片：`pipeline[1:3]` 返回子管道，便于测试部分流程。

统一处理不同数据：多模态数据（图像 + 关节状态 + 其他）

具体做了：

1. 重命名键
2. 图像转换，应用 `torchvision transforms`
3. 归一化
4. 设备转移（cpu/gpu）

3. 数据转换（从原始数据值（values）中构建一个标准化“帧”（frame）字典）

把散乱的原始观测/动作数据“打包”成一个 `numpy-based` 的 `dict`，确保类型、形状、前缀匹配数据集要求

```
def build_dataset_frame(
    ds_features: dict[str, dict], values: dict[str, Any], prefix: str
) -> dict[str, np.ndarray]:
    """根据数据集特征，从原始值构建单个数据框。
```

“frame”是一个字典，其中包含单个时间步的所有数据，

数据根据特征规范格式化为 NumPy 数组。

参数：

ds_features (字典)：LeRobot 数据集特征字典。

values (字典)：来自硬件/环境的原始值字典。

prefix (字符串)：用于筛选特征的前缀（例如，“observation”或“action”）。

返回值：

dict：表示单个数据框的字典。

"""

```
frame = {}
```

```
for key, ft in ds_features.items():
```

```
    if key in DEFAULT_FEATURES or not key.startswith(prefix):
```

```
        continue #跳过默认特征
```

```
    elif ft["dtype"] == "float32" and len(ft["shape"]) == 1: #处理 float32 类型的一维
        frame[key] = np.array([values[name] for name in ft["names"]], dtype=np.floa
```

```
    elif ft["dtype"] in ["image", "video"]: # 处理图像或视频类型 ("image" 或 "video")
        frame[key] = values[key.removeprefix(f"{prefix}.images.")]
```

```
return frame
```